
Requirements, Design and Tasks for rxnet

Reactive execution networks

2026-03-11

Contents

Requirements Document	7
Introduction	7
Glossary	7
Requirements	8
Requirement 1: Shared Synchronous Phase Model	8
Requirement 2: Context and Input Snapshot Handling	8
Requirement 3: Deferred Action Execution	8
Requirement 4: Generic Node Runtime API	9
Requirement 5: FSM Semantics	9
Requirement 6: FSM Input Projection in C	10
Requirement 7: Petri Net Semantics	10
Requirement 8: PN Data Validation and Error Behavior	11
Requirement 9: C Frontend Lifecycle and Memory Management	11
Requirement 10: Python Frontend API Surface	11
Requirement 11: Integration Patterns and Examples	12
Requirement 12: Execution Model Constraints	12
Design Document: rxnet	15
Overview	15
Architecture	16
High-Level Architecture	16
Component Architecture	16
Components and Interfaces	17
Core Runtime	17
Context	17
FSM Frontend	18
Petri Net Frontend	19
Host Integration Layer	20
Data Models	20
Runtime Model	20
FSM Model	21

PN Model	21
Correctness Properties	21
Property 1: Phase Ordering Determinism	22
Property 2: Snapshot Consistency Within Tick	22
Property 3: Deferred Action Isolation	22
Property 4: Deferred Queue Reset	22
Property 5: Runtime Node Contract Safety	22
Property 6: C Node Capacity Enforcement	22
Property 7: FSM First-Match Transition Rule	22
Property 8: FSM No-Match Stability	23
Property 9: FSM Deferred Action Behavior	23
Property 10: FSM Inputs Projector Invocation	23
Property 11: PN Transition Enablement	23
Property 12: PN Guard Enforcement	23
Property 13: PN Commit Delta Correctness	23
Property 14: PN Arc Validation in C	23
Property 15: PN Invalid Arc Exceptions in Python	24
Property 16: C Lifecycle Idempotent Safety	24
Property 17: C Context Ownership Boundary	24
Property 18: Python API Surface Stability	24
Property 19: Example Executability	24
Property 20: Host-Controlled Scheduling	24
Error Handling	24
Error Categories	24
Error Handling Strategies	25
Testing Strategy	25
Dual Testing Approach	25
Property-Based Testing Configuration	25
Integration Testing Strategy	26
Code Quality and Development Standards	26
C Standards	26
Python Standards	26
Compatibility Rules	26
Non-Goals and Out-of-Scope	27
Implementation Plan: rxnet	29
Overview	29
Tasks	29

Notes	33
-----------------	----

List of Figures

1	High-level architecture of the <code>rxnet</code> runtime and model frontends	16
---	---	----

Requirements Document

Introduction

This document specifies the requirements for `rxnet`, a reactive synchronous runtime library that supports two model families: Finite State Machines (FSM) and Petri Nets (PN). The system provides a shared phase-based execution core and language frontends in C and Python.

The goal of this document is to capture what the library provides today, so future changes can be validated and reflected here first.

Glossary

- **RXNET_System:** The complete `rxnet` library surface in C and Python
- **Core_Runtime:** Shared synchronous execution engine with phase-based ticking
- **Context:** Execution context containing live inputs, latched inputs, and deferred actions
- **Node:** Runtime unit with `evaluate` and `commit` phases
- **Deferred_Action:** Action scheduled during commit and executed after all commits
- **FSM_Runtime:** FSM frontend wrapper over the core runtime
- **FSM_Machine:** State machine node with transitions, guards, and optional actions
- **FSM_Transition:** Rule from `from_state` to `to_state`, with optional guard/action
- **Inputs_Projector:** C-only hook to map shared latched inputs into machine-local user data before FSM guard evaluation
- **PN_Runtime:** Petri Net frontend wrapper over the core runtime
- **PN_Net:** Petri net node with places, transitions, and firing flags
- **PN_Transition:** Petri transition with consume/produce arcs, optional guard/action
- **Arc:** Petri arc with `place_id` and `weight`
- **Tick:** One complete synchronous cycle: latch -> evaluate -> commit -> deferred actions
- **Latched_Inputs:** Immutable snapshot of current inputs used during one tick

Requirements

Requirement 1: Shared Synchronous Phase Model

User Story: As a model developer, I want deterministic phase-based execution, so that all nodes observe a consistent input snapshot and state updates happen predictably.

Acceptance Criteria

1. WHEN a tick starts, THE Core_Runtime SHALL latch input data before node evaluation
2. WHEN latching completes, THE Core_Runtime SHALL evaluate all registered nodes
3. WHEN evaluation completes, THE Core_Runtime SHALL commit all registered nodes
4. WHEN commit completes, THE Core_Runtime SHALL execute deferred actions and clear the queue
5. THE Tick order SHALL remain `latch -> evaluate -> commit -> deferred actions` in both C and Python implementations

Requirement 2: Context and Input Snapshot Handling

User Story: As an integrator, I want separate live and latched inputs, so that guards read a stable snapshot during each tick.

Acceptance Criteria

1. THE Context SHALL expose mutable live inputs for application/driver updates
2. THE Context SHALL expose latched inputs used during evaluation
3. WHEN ticking in C, THE runtime SHALL copy `inputs_size` bytes from `inputs` to `latched_inputs`
4. WHEN ticking in Python, THE runtime SHALL update `latched_inputs` using `copy.copy(inputs)`
5. THE C implementation SHALL keep ownership of the live inputs buffer in the application (runtime does not free it)

Requirement 3: Deferred Action Execution

User Story: As a runtime user, I want actions to run after all commits, so that side effects are deferred and model state commits are consistent.

Acceptance Criteria

1. WHEN a node commits with a proposed action, THE Context SHALL enqueue the action for deferred execution
2. THE Core_Runtime SHALL run deferred actions only after all node commits in the tick
3. THE deferred action queue SHALL be cleared after execution
4. THE C implementation SHALL grow deferred action storage dynamically when capacity is exceeded
5. WHEN enqueue parameters are invalid in C (`ctx` or function is null), THE API SHALL return an error code (-1)

Requirement 4: Generic Node Runtime API

User Story: As a library user, I want a generic runtime abstraction, so that different model families can share one execution engine.

Acceptance Criteria

1. THE Node contract SHALL include `evaluate` and `commit` callbacks
2. THE C runtime SHALL support runtime initialization with explicit node capacity
3. WHEN adding nodes in C beyond configured capacity, THE runtime SHALL reject the operation with -1
4. THE Python runtime SHALL support dynamic node list growth without explicit capacity configuration
5. WHEN `tick` is called with invalid runtime/context in C, THE API SHALL return -1

Requirement 5: FSM Semantics

User Story: As an FSM author, I want first-match transition evaluation with deferred actions, so that machine behavior is deterministic and clear.

Acceptance Criteria

1. WHEN an FSM machine evaluates, THE machine SHALL reset `next_state` to current state and clear proposed action
2. THE FSM evaluator SHALL scan transitions in declaration order and select the first transition whose `from_state` matches and whose guard is true (or absent)

3. WHEN no transition matches, THE machine SHALL remain in the current state
4. WHEN committing, THE machine SHALL update current state to `next_state`
5. WHEN a transition action exists, THE action SHALL be enqueued as a deferred action (not executed inline during evaluate/commit)

Requirement 6: FSM Input Projection in C

User Story: As an embedded integrator, I want optional per-machine input projection, so that multiple machines can share one global input snapshot while evaluating machine-local signals.

Acceptance Criteria

1. THE C FSM API SHALL support configuring an optional `inputs_projector` callback per machine
2. WHEN `inputs_projector` is configured, THE callback SHALL execute at the start of machine evaluation
3. THE projector SHALL receive the shared latched context and machine user payload
4. THE runtime SHALL allow multiple FSM machines to share one context input struct
5. IF no projector is configured, THEN FSM evaluation SHALL proceed directly with transition guard checks

Requirement 7: Petri Net Semantics

User Story: As a Petri Net author, I want place/transition execution with guards and arc validation, so that token flow is modeled consistently.

Acceptance Criteria

1. WHEN a PN net evaluates, THE net SHALL copy `places` into `next_places` and reset transition fire flags
2. THE PN evaluator SHALL mark a transition as fireable only if consume/produce arcs are valid and consume tokens are available
3. WHEN a transition guard exists, THE transition SHALL fire only if the guard returns true
4. WHEN committing, THE net SHALL apply consume/produce deltas for each fire-flagged transition
5. WHEN a transition action exists, THE action SHALL be enqueued as a deferred action

Requirement 8: PN Data Validation and Error Behavior

User Story: As a developer, I want invalid Petri model data to be detected, so that configuration errors fail early.

Acceptance Criteria

1. WHEN initializing a PN net in C, THE API SHALL validate all consume/produce arc lists and reject invalid models with -1
2. THE C PN arc validator SHALL reject arcs with out-of-range `place_id` or negative `weight`
3. WHEN PN net allocation fails in C, THE API SHALL free partial allocations and return -1
4. IN Python PN evaluation, invalid arc `place_id` SHALL raise `IndexError`
5. IN Python PN evaluation, negative arc weights SHALL raise `ValueError`

Requirement 9: C Frontend Lifecycle and Memory Management

User Story: As a C integrator, I want explicit initialization and cleanup APIs, so that runtime memory is managed safely.

Acceptance Criteria

1. THE C API SHALL provide runtime init/free functions for both FSM and PN frontends
2. THE C API SHALL provide model registration functions (`add_machine`, `add_net`) before ticking
3. THE C API SHALL provide model-specific cleanup for PN net internal buffers (`rx_pn_net_free`)
4. WHEN freeing runtime/context in C, THE API SHALL reset internal pointers and counters to neutral values
5. WHEN null pointers are passed to C free functions, THE functions SHALL return safely without crashing

Requirement 10: Python Frontend API Surface

User Story: As a Python developer, I want ergonomic dataclass-based APIs, so that models are easy to define and run.

Acceptance Criteria

1. THE Python FSM frontend SHALL expose `Machine` and `Transition` dataclasses with optional guard/action callbacks
2. THE Python PN frontend SHALL expose `Net`, `Transition`, and `Arc` dataclasses
3. THE Python runtime wrappers (`rxnet.fsm.Runtime`, `rxnet.pn.Runtime`) SHALL expose `context`, model registration, and `tick()`
4. THE root package SHALL expose `fsm`, `pn`, `runtime`, and FSM-oriented aliases currently defined in `rxnet.__init__`
5. THE Python implementation SHALL rely only on standard-library modules present in the code-base (`dataclasses`, `typing`, `copy`)

Requirement 11: Integration Patterns and Examples

User Story: As a user evaluating the library, I want runnable examples in C and Python, so that I can understand expected integration patterns.

Acceptance Criteria

1. THE repository SHALL include runnable C examples for FSM and PN under `c/examples`
2. THE repository SHALL include runnable Python examples for FSM and PN under `python/examples`
3. THE example structure in Python SHALL separate model definition, system input writer, and runtime wiring (`model.py`, `system.py`, `main.py`)
4. THE repository SHALL include an ESP-IDF FSM integration example with periodic ticking and ISR-driven input updates
5. THE host CLI FSM example (`c/examples/fsm/00-light/main_cli.c`) SHALL support interactive commands to trigger input events and ticks

Requirement 12: Execution Model Constraints

User Story: As a platform engineer, I want clear operational constraints, so that I can integrate `rxnet` correctly in larger systems.

Acceptance Criteria

1. THE runtime SHALL be single-threaded per runtime instance unless external synchronization is provided by the integrator
2. THE library SHALL not include built-in networking, persistence, or REST APIs
3. THE library SHALL leave scheduling policy (tick frequency/loop ownership) to the host application
4. THE library SHALL leave I/O side effects to user-defined action callbacks
5. THE library SHALL support using one shared typed input structure across multiple model nodes in the same runtime context

Design Document: rxnet

Overview

This document outlines the design for `rxnet`, a synchronous reactive runtime library with two model frontends (Finite State Machine and Petri Net) built on a shared phase-based core.

`rxnet` is implemented in C and Python with equivalent execution semantics. The core design decision is to centralize tick orchestration (`latch` -> `evaluate` -> `commit` -> `deferred actions`) and let each model frontend implement only model-specific behavior on top of a generic node contract.

The design emphasizes deterministic execution, explicit input snapshotting, deferred side effects, and lightweight integration into host loops (CLI, embedded, or application-managed schedulers).

Architecture

High-Level Architecture

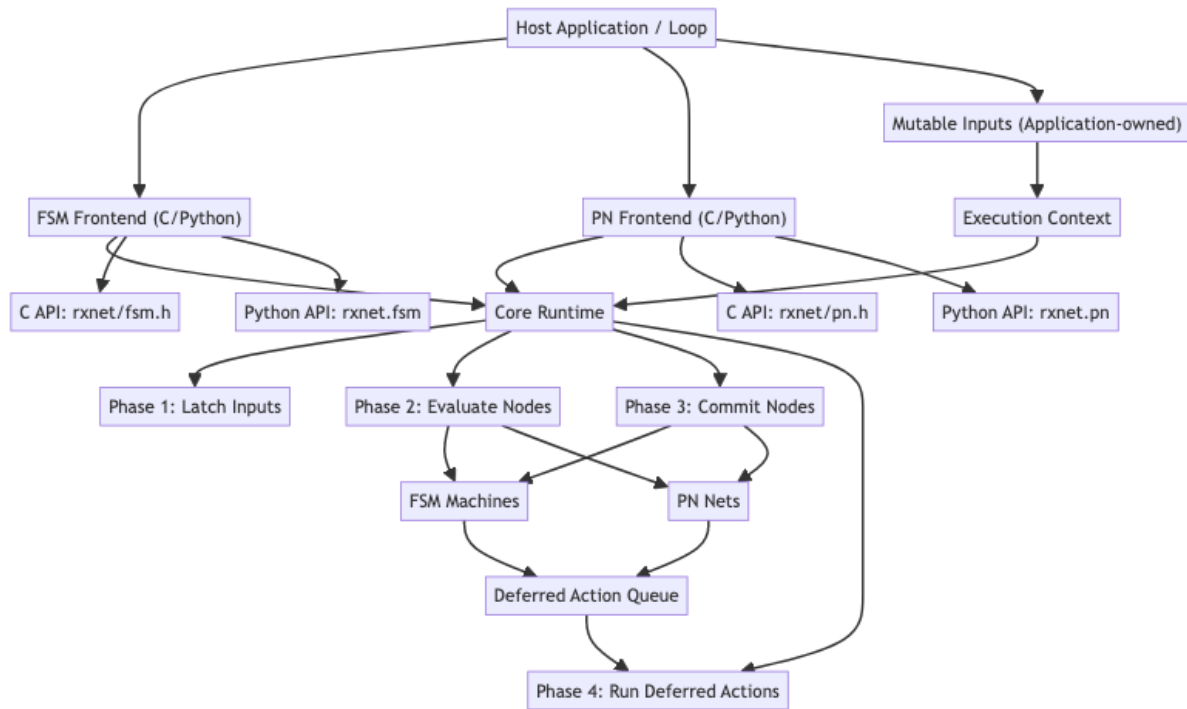


Figure 1: High-level architecture of the `rxnet` runtime and model frontends

Component Architecture

The system follows a layered runtime architecture:

1. **Host Integration Layer:** Application-owned inputs, tick scheduling, and output side effects
2. **Model Frontends:** FSM and PN model APIs (C and Python)
3. **Core Runtime Layer:** Generic node container and phase-ordered tick execution
4. **Execution Context Layer:** Live inputs, latched snapshot, deferred action queue
5. **Model Nodes:** Concrete node implementations (`Machine`, `Net`) with evaluate/commit logic

Components and Interfaces

Core Runtime

Responsibilities: - Maintain a list of executable nodes - Execute deterministic tick phases - Coordinate context latch and deferred actions - Provide minimal lifecycle surface (`init/add/tick/free`)

Key Interfaces (C):

```

1 typedef struct rx_node_vtable {
2     void (*evaluate)(void *node, rx_context *ctx);
3     void (*commit)(void *node, rx_context *ctx);
4 } rx_node_vtable;
5
6 typedef struct rx_node {
7     const rx_node_vtable *vtable;
8     void *self;
9 } rx_node;
10
11 int rx_runtime_init(rx_runtime *rt, rx_context *ctx, size_t
    node_capacity);
12 int rx_runtime_add_node(rx_runtime *rt, rx_node node);
13 int rx_tick(rx_runtime *rt);
14 void rx_runtime_free(rx_runtime *rt);

```

Key Interfaces (Python):

```

1 class Node(Protocol):
2     def evaluate(self, ctx: Context) -> None: ...
3     def commit(self, ctx: Context) -> None: ...
4
5 class Runtime:
6     def add_node(self, node: Node) -> None: ...
7     def tick(self) -> None: ...

```

Context

Responsibilities: - Hold mutable application inputs (`inputs`) - Hold immutable per-tick snapshot (`latched_inputs`) - Queue and execute deferred actions

Design Notes: - C latching is byte-copy (`memcpy`) using explicit `inputs_size` - Python latching is shallow copy (`copy.copy`) - Live input ownership remains external in C

Key Interfaces:

```

1 int rx_context_init(rx_context *ctx, void *inputs, size_t inputs_size);
2 void rx_contextLatch_inputs(rx_context *ctx);

```

```

3  int rx_context_enqueue_deferred_action(rx_context *ctx,
    rx_deferred_action_fn fn, void *user);
4  void rx_context_run_deferred_actions(rx_context *ctx);
5  void rx_context_free(rx_context *ctx);

```

```

1  class Context:
2      inputs: Any
3      latched_inputs: Any
4      def latch_inputs(self) -> None: ...
5      def enqueue_deferred_action(self, fn: Any, user: Any) -> None: ...
6      def run_deferred_actions(self) -> None: ...

```

FSM Frontend

Responsibilities: - Define machine transitions (*from_state*, *to_state*, *guard*, *action*) - Evaluate first valid transition in declaration order - Commit next state and enqueue optional deferred action - Support optional C inputs projection hook per machine

Key Interfaces (C):

```

1  typedef int (*rx_fsm_guard_fn)(const rx_fsm_context *ctx, void *user);
2  typedef void (*rx_fsm_action_fn)(rx_fsm_context *ctx, void *user);
3  typedef void (*rx_fsm_inputs_projector_fn)(const rx_fsm_context *ctx,
    void *user);
4
5  void rx_fsm_machine_init(
6      rx_fsm_machine *machine,
7      const char *name,
8      int initial_state,
9      const rx_fsm_transition *transitions,
10     size_t transition_count,
11     void *user
12 );
13 void rx_fsm_machine_set_inputs_projector(rx_fsm_machine *machine,
    rx_fsm_inputs_projector_fn projector);
14 int rx_fsm_runtime_add_machine(rx_fsm_runtime *runtime, rx_fsm_machine
    *machine);
15 int rx_fsm_tick(rx_fsm_runtime *runtime);

```

Key Interfaces (Python):

```

1  @dataclass(frozen=True, slots=True)
2  class Transition:
3      from_state: int
4      to_state: int
5      guard: Optional[Guard] = None
6      action: Optional[Action] = None
7

```

```

8 @dataclass(slots=True)
9 class Machine:
10     name: str
11     state: int
12     transitions: Sequence[Transition]
13     user: Any = None

```

Petri Net Frontend

Responsibilities: - Represent places and transitions with consume/produce arcs - Evaluate transition enablement by token availability and guards - Apply transition deltas in commit phase - Enqueue transition actions as deferred side effects

Key Interfaces (C):

```

1 typedef struct rx_pn_arc {
2     size_t place_id;
3     int weight;
4 } rx_pn_arc;
5
6 typedef struct rx_pn_transition {
7     const rx_pn_arc *consume;
8     size_t consume_count;
9     const rx_pn_arc *produce;
10    size_t produce_count;
11    rx_pn_guard_fn guard;
12    rx_pn_action_fn action;
13 } rx_pn_transition;
14
15 int rx_pn_net_init(
16     rx_pn_net *net,
17     const char *name,
18     const int *initial_places,
19     size_t place_count,
20     const rx_pn_transition *transitions,
21     size_t transition_count,
22     void *user
23 );
24 int rx_pn_runtime_add_net(rx_pn_runtime *runtime, rx_pn_net *net);
25 int rx_pn_tick(rx_pn_runtime *runtime);

```

Key Interfaces (Python):

```

1 @dataclass(frozen=True, slots=True)
2 class Arc:
3     place_id: int
4     weight: int = 1
5

```

```

6 @dataclass(frozen=True, slots=True)
7 class Transition:
8     consume: Sequence[Arc] = ()
9     produce: Sequence[Arc] = ()
10    guard: Optional[Guard] = None
11    action: Optional[Action] = None
12
13 @dataclass(slots=True)
14 class Net:
15     name: str
16     places: Sequence[int]
17     transitions: Sequence[Transition]
18     user: Any = None

```

Host Integration Layer

Responsibilities: - Own and mutate live inputs before each tick - Invoke tick in a periodic or event-driven loop - Reset edge-triggered inputs when required by domain logic - Implement side effects in action callbacks

Supported Patterns (as implemented in examples): - Host CLI loop for manual event injection and status inspection - Embedded periodic loop (ESP-IDF `vTaskDelayUntil`) with ISR-updated inputs - Python script loops with explicit input driver objects

Data Models

Runtime Model

```

1 interface RuntimeModel {
2     context: ContextModel
3     nodes: NodeModel[]
4     tickPhases: ['latch', 'evaluate', 'commit', 'deferred']
5 }
6
7 interface ContextModel {
8     inputs: unknown
9     latchedInputs: unknown
10    deferredActions: DeferredActionModel[]
11 }
12
13 interface DeferredActionModel {
14     fn: Function
15     user: unknown
16 }

```

FSM Model

```
1 interface FSMMachineModel {
2     name: string
3     state: number
4     nextState: number
5     transitions: FSMTransitionModel[]
6     user?: unknown
7 }
8
9 interface FSMTransitionModel {
10     fromState: number
11     toState: number
12     guard?: (ctx: unknown, user: unknown) => boolean
13     action?: (ctx: unknown, user: unknown) => void
14 }
```

PN Model

```
1 interface PNNetModel {
2     name: string
3     places: number[]
4     nextPlaces: number[]
5     transitions: PNTransitionModel[]
6     fireFlags: boolean[]
7     user?: unknown
8 }
9
10 interface PNTransitionModel {
11     consume: PNArcModel[]
12     produce: PNArcModel[]
13     guard?: (ctx: unknown, user: unknown) => boolean
14     action?: (ctx: unknown, user: unknown) => void
15 }
16
17 interface PNArcModel {
18     placeId: number
19     weight: number
20 }
```

Correctness Properties

A property is a behavior that should hold for all valid executions. Properties connect requirements with verifiable guarantees in unit, integration, and property-based tests.

Property 1: Phase Ordering Determinism

For any tick execution, phase order should always be `latch` -> `evaluate` -> `commit` -> `deferred`. **Validates: Requirements 1.1, 1.2, 1.3, 1.4, 1.5**

Property 2: Snapshot Consistency Within Tick

For any guard evaluation during one tick, observed inputs should come from `latched_inputs` and remain stable for that tick. **Validates: Requirements 2.1, 2.2, 2.3, 2.4**

Property 3: Deferred Action Isolation

For any action emitted during commit, execution should occur only after all nodes have completed commit. **Validates: Requirements 3.1, 3.2**

Property 4: Deferred Queue Reset

For any completed tick, deferred queue length should be zero after running deferred actions. **Validates: Requirements 3.3**

Property 5: Runtime Node Contract Safety

For any node registered in the runtime, `evaluate` and `commit` should both be callable in every tick. **Validates: Requirements 4.1**

Property 6: C Node Capacity Enforcement

For any C runtime initialized with capacity `N`, adding more than `N` nodes should fail with `-1`. **Validates: Requirements 4.2, 4.3**

Property 7: FSM First-Match Transition Rule

For any FSM machine and state, transition selection should be the first declaration-order transition that matches state and guard. **Validates: Requirements 5.1, 5.2**

Property 8: FSM No-Match Stability

*For any FSM tick with no valid transition, machine state should remain unchanged. **Validates: Requirements 5.3***

Property 9: FSM Deferred Action Behavior

*For any matched FSM transition with action, the action should be queued for deferred execution, not run inline. **Validates: Requirements 5.5***

Property 10: FSM Inputs Projector Invocation

*For any C FSM machine with configured projector, projector should run before transition guard checks. **Validates: Requirements 6.1, 6.2, 6.3***

Property 11: PN Transition Enablement

*For any PN transition, firing should require valid arcs and sufficient consume tokens. **Validates: Requirements 7.2***

Property 12: PN Guard Enforcement

*For any enabled PN transition with guard, transition should fire only when guard is true. **Validates: Requirements 7.3***

Property 13: PN Commit Delta Correctness

*For any set of fireable PN transitions, place deltas should match arc consume/produce sums in commit. **Validates: Requirements 7.4***

Property 14: PN Arc Validation in C

*For any C PN net init with invalid arc index or negative weight, init should fail with -1. **Validates: Requirements 8.1, 8.2***

Property 15: PN Invalid Arc Exceptions in Python

*For any Python PN evaluation with invalid arc data, runtime should raise typed exceptions (`IndexError` / `ValueError`). **Validates: Requirements 8.4, 8.5***

Property 16: C Lifecycle Idempotent Safety

*For any `*_free(NULL)` call in C APIs, operation should be safe and not crash. **Validates: Requirements 9.5***

Property 17: C Context Ownership Boundary

*For any C runtime lifecycle, external input buffer should not be freed by runtime/context cleanup. **Validates: Requirements 2.5, 9.4***

Property 18: Python API Surface Stability

*For any supported Python import path, package exports and runtime wrappers should match documented API. **Validates: Requirements 10.1, 10.2, 10.3, 10.4***

Property 19: Example Executability

*For any provided example entrypoint, example should run and complete without runtime errors in a valid environment/toolchain. **Validates: Requirements 11.1, 11.2, 11.3, 11.4, 11.5***

Property 20: Host-Controlled Scheduling

*For any deployment context, tick frequency and loop ownership should remain controlled by host code, not by runtime internals. **Validates: Requirements 12.3***

Error Handling**Error Categories**

1. API Misuse Errors (C return-code based) - Null pointers in required parameters - Capacity overflow in node registration - Invalid PN initialization data (arcs, buffers)

2. Allocation and Resource Errors (C) - Context buffer allocation failure - Deferred queue growth failure - Net internal array allocation failure

3. Model Validation Errors (Python exception based) - PN invalid `place_id` (`IndexError`) - PN negative arc weight (`ValueError`)

4. Integration Errors - Host passes invalid input lifetimes - Host loop omits input reset logic for edge-triggered signals

Error Handling Strategies

C Strategy: explicit status codes - Initialization and tick APIs return 0 on success, -1 on failure - Cleanup APIs are null-safe and best-effort - Partial init failures trigger cleanup of already allocated resources

Python Strategy: fail-fast typed exceptions - Invalid PN arc data raises typed exceptions during evaluation - User callbacks may raise and should be handled by host application policy

Host Strategy: explicit loop ownership - Host checks return codes/exceptions per tick - Host applies retry/abort policy appropriate to runtime context (CLI/embedded/service)

Testing Strategy

Dual Testing Approach

`rxnet` should combine deterministic example tests, unit tests, and property-based tests:

Unit Tests focus on: - Per-function semantics in runtime/fsm/pn layers - Edge cases (empty transitions, zero-capacity runtime, invalid arcs) - Correct deferred action behavior

Property Tests focus on: - Tick-order invariants - Equivalence of C and Python behavioral semantics for shared scenarios - PN token conservation and transition correctness across randomized nets - FSM deterministic transition ordering across randomized transition lists

Property-Based Testing Configuration

Frameworks: - Python: `hypothesis` - C: randomized harness plus deterministic replay seeds

Configuration guidelines: - Minimum 100 runs per property - Each property references its design property number - Seeded reproducibility for failure triage

Example Property Test Structure (Python):

```
1 # Property 7: FSM First-Match Transition Rule
2 @given(st.lists(transition_strategy, min_size=1, max_size=20))
3 def test_fsm_first_match_ordering(transitions):
4     machine = Machine(name="m", state=0, transitions=transitions)
5     rt = Runtime(inputs=DummyInputs())
6     rt.add_machine(machine)
7     rt.tick()
8     # Assert selected transition corresponds to first matching
       transition
```

Integration Testing Strategy

Cross-layer integration: - Host inputs mutation -> runtime tick -> state update -> deferred side effects
- Multiple nodes sharing one context input struct - C FSM projector-based input mapping with shared global input

Environment integration: - C host example builds (gcc) for FSM and PN - ESP-IDF example compile-level validation - Python example execution paths

Code Quality and Development Standards

C Standards

- Compile with `-std=c11 -Wall -Wextra -Wpedantic` (as used in examples)
- Preserve null-safety checks and explicit error returns
- Keep ownership boundaries explicit (runtime-owned vs host-owned memory)

Python Standards

- Use type hints and dataclasses consistently (current implementation pattern)
- Keep runtime/frontend modules dependency-light (standard library only in core)
- Preserve API compatibility for existing imports from `rxnet.__init__`

Compatibility Rules

- Maintain semantic parity of tick phases across C and Python
- Additive API evolution preferred; breaking changes require requirements/design updates first
- Example programs are part of behavioral contract and should be updated with API changes

Non-Goals and Out-of-Scope

- No built-in networking or REST API layer
- No built-in persistence, serialization, or storage backends
- No built-in scheduler, thread pool, or realtime policy manager
- No automatic synchronization for concurrent access to one runtime instance

These concerns are intentionally delegated to host applications.

Implementation Plan: rxnet

Overview

This implementation plan converts the [rxnet](#) design into discrete tasks for completing and validating the library. It follows an incremental approach: core runtime semantics first, model frontends second, integration examples third, and verification/quality automation last.

Tasks are marked as completed when they reflect the current repository state, and left pending where implementation, tests, or automation are still missing.

Tasks

- ☒ 1. Establish repository structure and multi-language layout
 - ☒ 1.1 Create C runtime/frontend directories and public headers
 - ★ Add `c/include/rxnet` and `c/src` layout for core runtime + FSM + PN
 - ★ *Requirements: 4.1, 9.1*
 - ☒ 1.2 Create Python package layout and modules
 - ★ Add `python/rxnet` package with `runtime.py`, `fsm.py`, `pn.py`, `__init__.py`
 - ★ *Requirements: 10.1, 10.2, 10.3, 10.4*
 - ☒ 1.3 Add top-level and per-language README docs
 - ★ Document phase model and basic execution usage
 - ★ *Requirements: 11.1, 11.2*
- ☒ 2. Implement shared synchronous core runtime (C)
 - ☒ 2.1 Implement context lifecycle and input latching
 - ★ Add `rx_context_init/free/latch_inputs`
 - ★ Implement `inputs` + `latched_inputs` model and ownership boundaries
 - ★ *Requirements: 1.1, 2.1, 2.2, 2.3, 2.5, 9.4*

- ☒ 2.2 Implement deferred action queue
 - * Add enqueue/run APIs with dynamic capacity growth
 - * *Requirements: 3.1, 3.2, 3.3, 3.4, 3.5*
- ☒ 2.3 Implement generic runtime node orchestration
 - * Add runtime init/add/tick/free and phase ordering
 - * *Requirements: 1.2, 1.3, 1.4, 1.5, 4.1, 4.2, 4.3, 4.5*
- ☒ 3. Implement FSM frontend (C)
 - ☒ 3.1 Implement machine model and transition semantics
 - * First-match transition selection in declaration order
 - * Commit state and enqueue deferred action
 - * *Requirements: 5.1, 5.2, 5.3, 5.4, 5.5*
 - ☒ 3.2 Implement FSM runtime wrapper
 - * Runtime init/free, machine registration, tick wrapper
 - * *Requirements: 9.1, 9.2*
 - ☒ 3.3 Implement optional inputs projector hook
 - * Add per-machine projector called before guard evaluation
 - * *Requirements: 6.1, 6.2, 6.3, 6.4, 6.5*
- ☒ 4. Implement Petri Net frontend (C)
 - ☒ 4.1 Implement PN net model and transition execution
 - * Evaluate fire flags from token availability + guards
 - * Apply consume/produce deltas during commit
 - * *Requirements: 7.1, 7.2, 7.3, 7.4, 7.5*
 - ☒ 4.2 Implement PN validation and allocation safety
 - * Validate arcs and reject invalid models
 - * Handle allocation failures with cleanup
 - * *Requirements: 8.1, 8.2, 8.3*
 - ☒ 4.3 Implement PN runtime wrapper and lifecycle APIs
 - * Runtime init/free, net add/tick, net free
 - * *Requirements: 9.1, 9.2, 9.3, 9.5*
- ☒ 5. Implement shared synchronous core runtime (Python)
 - ☒ 5.1 Implement Python context with latched input snapshot
 - * Use `copy.copy` snapshot semantics
 - * *Requirements: 2.1, 2.2, 2.4*

- ☒ 5.2 Implement Python deferred action queue
 - * Enqueue and execute post-commit actions
 - * *Requirements: 3.1, 3.2, 3.3*
- ☒ 5.3 Implement Python generic runtime orchestration
 - * Node registration and deterministic phase-ordered tick
 - * *Requirements: 1.1, 1.2, 1.3, 1.4, 1.5, 4.1, 4.4*
- ☒ 6. Implement FSM and PN frontends (Python)
 - ☒ 6.1 Implement dataclass-based FSM API
 - * `Machine` and `Transition` with first-match semantics
 - * *Requirements: 5.1, 5.2, 5.3, 5.4, 5.5, 10.1*
 - ☒ 6.2 Implement dataclass-based PN API
 - * `Net`, `Transition`, `Arc` and commit delta application
 - * *Requirements: 7.1, 7.2, 7.3, 7.4, 7.5, 10.2*
 - ☒ 6.3 Implement Python validation behavior
 - * Raise `IndexError` for out-of-range `place_id`
 - * Raise `ValueError` for negative arc weight
 - * *Requirements: 8.4, 8.5*
 - ☒ 6.4 Expose package API aliases
 - * Export modules and FSM-oriented aliases in `rxnet.__init__`
 - * *Requirements: 10.3, 10.4, 10.5*
- ☒ 7. Provide runnable examples and host integration patterns
 - ☒ 7.1 Implement C FSM and PN console examples
 - * Add deterministic sample flows for state/token updates
 - * *Requirements: 11.1, 12.3, 12.4*
 - ☒ 7.2 Implement C FSM 00-light host CLI and ESP-IDF variants
 - * Shared input struct + machine-local projection pattern
 - * *Requirements: 6.4, 11.4, 11.5, 12.3, 12.5*
 - ☒ 7.3 Implement Python FSM and PN examples
 - * Split model/system/main integration structure
 - * *Requirements: 11.2, 11.3, 12.3, 12.5*
- ☒ 8. Create baseline specification documents
 - ☒ 8.1 Write requirements specification
 - * Add `docs/specs/requirements.md` as source of truth for behavior

- * *Requirements: All requirements traceability*
- ☒ 8.2 Write design specification
 - * Add `docs/specs/design.md` with architecture, interfaces, and properties
 - * *Requirements: All requirements traceability*
- ☐ 9. Add C unit and integration tests
 - ☐ 9.1 Add runtime/context tests
 - * Tick ordering, deferred queue, capacity limits, null-safety checks
 - * *Requirements: 1.1, 1.5, 3.3, 4.3, 4.5, 9.5*
 - ☐ 9.2 Add FSM semantic tests
 - * First-match ordering, projector invocation, no-match stability
 - * *Requirements: 5.2, 5.3, 6.2*
 - ☐ 9.3 Add PN semantic/validation tests
 - * Arc validation, token constraints, delta correctness, net lifecycle
 - * *Requirements: 7.2, 7.4, 8.1, 8.2, 9.3*
- ☐ 10. Add Python unit and property-based tests
 - ☐ 10.1 Add runtime/frontend unit tests
 - * Cover FSM and PN deterministic behavior and error paths
 - * *Requirements: 1.5, 5.2, 7.2, 8.4, 8.5*
 - [* 10.2 Add property tests for correctness properties
 - * **Property 1: Phase Ordering Determinism**
 - * **Property 7: FSM First-Match Transition Rule**
 - * **Property 13: PN Commit Delta Correctness**
 - * **Property 15: PN Invalid Arc Exceptions in Python**
 - * *Requirements: 1.1, 5.2, 7.4, 8.4, 8.5*
- ☐ 11. Add C/Python semantic parity test harness
 - ☐ 11.1 Define shared scenario corpus
 - * Scenarios for FSM and PN with expected traces
 - * *Requirements: 1.5, 12.5*
 - ☐ 11.2 Implement cross-language conformance runner
 - * Execute equivalent scenarios in C and Python and compare traces
 - * *Requirements: 1.5, 10.4, 12.1*
- ☐ 12. Implement developer quality automation

- ☐ 12.1 Add formatting/linting/type-check tools for Python
 - * Configure `black`, `isort`, `flake8`, and `mypy`
 - * *Requirements: 10.5 (maintainable API surface)*
- ☐ 12.2 Add C static analysis and warning gates
 - * Enforce warning-clean build in CI
 - * *Requirements: 9.4, 12.1*
- ☐ 12.3 Add pre-commit and CI checks
 - * Ensure docs + tests + quality gates run on every PR
 - * *Requirements: 11.1, 11.2*
- ☐ 13. Add packaging and release workflow
 - ☐ 13.1 Define Python packaging metadata and install flow
 - * Add `pyproject.toml` and package build configuration
 - * *Requirements: 10.3, 11.2*
 - ☐ 13.2 Define C library build/install workflow
 - * Add canonical build targets for library + examples
 - * *Requirements: 11.1, 12.3*
 - ☐ 13.3 Document versioning and compatibility rules
 - * Tie release notes to requirements/design deltas
 - * *Requirements: 12.1*
- ☐ 14. Final validation and readiness checkpoint
 - ☐ 14.1 Verify all correctness properties are covered by tests
 - * Map tests to properties in `docs/specs/design.md`
 - * *Requirements: All requirements*
 - ☐ 14.2 Verify requirements-to-implementation traceability
 - * Confirm every requirement has implemented code + tests or explicit backlog status
 - * *Requirements: All requirements*
 - ☐ 14.3 Establish “requirements-first” change workflow
 - * PR checklist rule: update `requirements.md` before behavioral code changes
 - * *Requirements: All requirements governance*

Notes

- Tasks marked with * are optional for early MVP but recommended for robustness.

- This file tracks implementation status against current repository state as of March 11, 2026.
- Any behavior change should first update [docs/specs/requirements.md](#), then [docs/specs/design.md](#), then this task plan.